



K. J. Somaiya College of Engineering, Mumbai-77

Batch: A-4 Roll No.:16010122151

Experiment / Assignment / Tutorial No: 3

TITLE Represent software architecture of B Tech. project in two known different styles

AIM: Understand the architecture of software systems in various architectural styles

Expected OUTCOME of Experiment:

Need of ADL and use of the same in representing software architecture

CO 1. Design the architecture of software systems in various architectural styles.

Books/ Journals/ Websites referred:

- www.isr.uci.edu/projects/archstudio/setup-easy.html

Background Theory:

You have studied various software architectural styles and patterns used in designing any software system. The style and pattern are based on the domain in which application is to be used and the approach of the organization.

You have identified the topic for your final year project.

Laboratory Work:

Students are required to identify and draw **TWO** different architectural diagram of the software system you are planning to implement. These two approaches to implement will decide the trade-off between various non-functional requirements

Title of LY Project: ScholarBridge — one-stop scholarship search & disbursement management platform

Abstract of Final year Project:

ScholarBridge is a web platform that connects students in need of scholarships with philanthropic trusts. Students register with verified identity and bank details, submit a scholarship application (education, family income, supporting documents). Registered trusts review applications, approve/reject and record approved amounts. Students



K. J. Somaiya College of Engineering, Mumbai-77

confirm receipt. SuperAdmin handles verification of trusts, resolves disputes, and generates analytics and historical reports. The system enforces yearly application frequency limits, secure document storage, and provides exportable PDFs and Excel reports for record-keeping.

Architecture 1 of Project:

The architecture is primarily **three-tier (presentation → application → data)** with service-oriented interfaces to external verification systems (DigiLocker, PAN e-verify providers). The system is implemented as a modern web app: React frontend (Student / Trust / Admin dashboards), Node/Express backend (business logic, APIs), a relational database (Postgres), and cloud document storage (S3/Cloudinary). External verification integrations are separated as external adapters/services. Background workers handle scheduled tasks (yearly closures, notifications, exports).

Explanation of Components and Connectors:

Components:

- **Student Portal (frontend)** — React SPA for student workflows: registration, application creation, document upload, receipt confirmation.
- **Trust Dashboard (frontend)** — React or separate UI for trust users: review apps, approve/reject, record payments, export reports.
- **Admin Dashboard (frontend)** — SuperAdmin UI for verification, issue resolution, analytics.
- **Scholarship Management Backend (application server)** — Node/Express (or your preferred backend) that implements authentication, RBAC, application workflow, business rules, PDF generation and connectors to document store & external verification APIs.
- **Application Database** — Relational DB (Postgres) storing user profiles, applications, approvals, payments, audit logs.
- **Document Storage** — Cloud storage (S3/Cloudinary) for uploaded documents (KYC, receipts).
- **Verification Adapters** — Integration modules for DigiLocker, PAN verification vendors, NGO Darpan lookups.
- **Worker / Scheduler** — Background worker (e.g., cron job/queue workers using Bull or Celery) for scheduled tasks (year-end checks, reminders, exports).
- **Analytics/Reporting Service** — Generates trust-specific charts & aggregated statistics; generates Excel/CSV exports.



K. J. Somaiya College of Engineering, Mumbai-77

- **Notification Service** — Wraps email/SMS/WhatsApp providers.

Connectors :

- **HTTPS / REST**: Frontends ↔ Backend (JSON REST / GraphQL).
- **DB Access (SQL)**: Backend ↔ Database.
- **Signed URLs**: Backend ↔ Document Storage (S3/Cloudinary).
- **OAuth / REST**: Backend ↔ DigiLocker / PAN providers / NGO Darpan (external APIs).
- **Message Queue**: Backend ↔ Worker (for background tasks such as PDF generation, scheduled checks).
- **TLS** for all external connections; use KMS for key management.

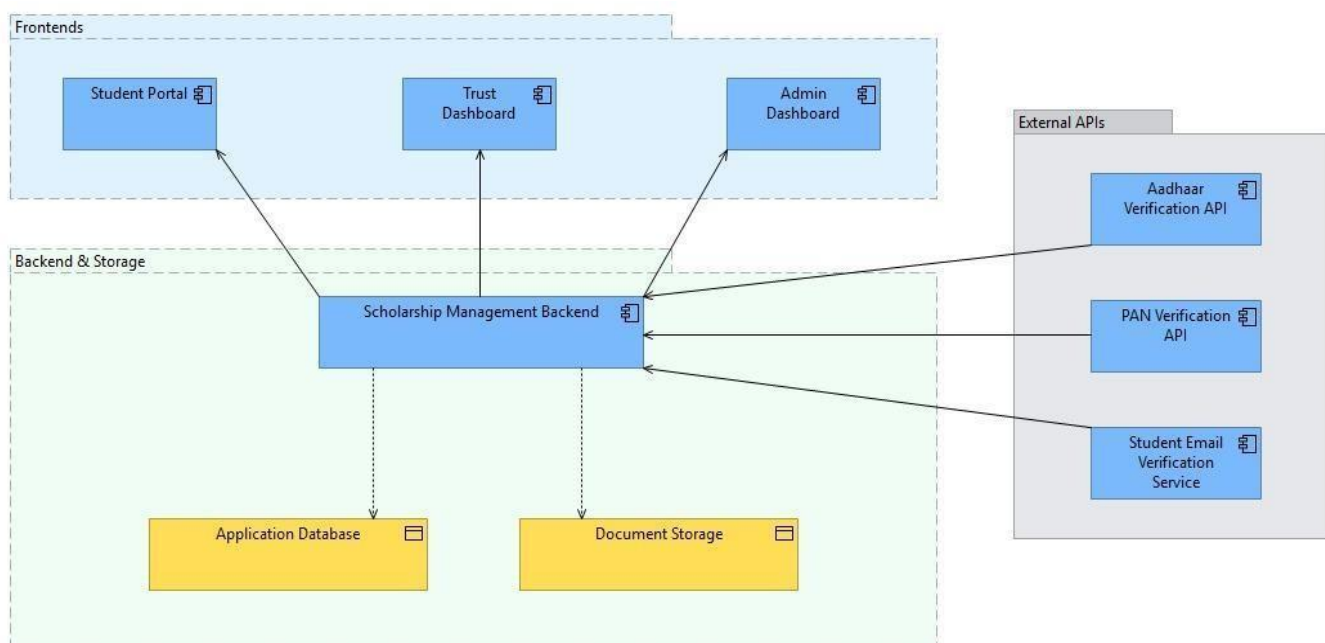
Architecture styles used in Project :

Style Name 1: Layered (n-tier / 3-tier) architecture — *Client / Server / Data*

Justification for Style-1:

Clear separation of concerns; matches the natural decomposition of a web app (UI, business logic, persistence). Easy for a student team to implement and test. Works well with MERN/PERN stack and is easy to document in Archi.

Straightforward to explain in diagrams, easy to implement in a team, simple testing and separation of UI/backend/DB. Best choice for MVP and final-year project.

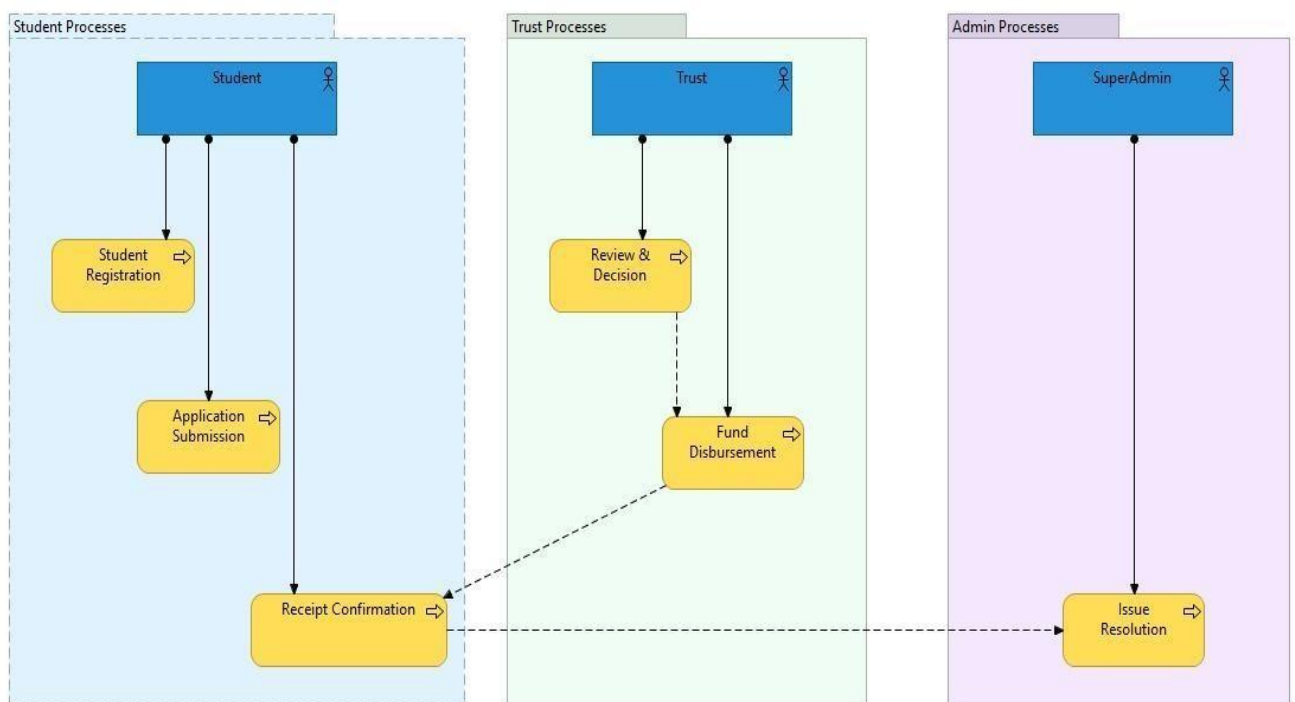


Style Name 2: Service-Oriented / Component-and-Connector style (a service-oriented variant / microservices-like)

Justification for Style-2:

Parts of ScholarBridge (verification, PDF generation, analytics) map well to independent services (Verification Adapter Service, PDF Service, Analytics Service). This gives independent deployability and isolation for sensitive components (e.g., a separate verification service that handles Aadhaar/DigiLocker calls and special security requirements).

Useful when you want to isolate compliance-sensitive parts (Aadhaar verification, PAN checks) into separate deployable services with stricter infrastructure/security. Makes scaling specific parts easier.



Post Laboratory questions:

1. What is the design vocabulary?

- Design vocabulary refers to the comprehensive and cohesive set of visual and functional elements that define the style and identity of a product, system, or brand. It acts like a language comprised of specific shapes, lines, colors, textures, typography, patterns, and spatial arrangements that designers use consistently to communicate a recognizable visual and experiential message.



K. J. Somaiya College of Engineering, Mumbai-77

- This vocabulary is not just a random set of design features; it's a carefully curated toolkit that ensures harmony and alignment across all design outputs within a particular domain or product family. For instance, in user interface design, a design vocabulary could include standardized button shapes, icon styles, color palettes, font families, spacing rules, and motion behaviors that together create a seamless user experience and brand harmony.
- Importantly, design vocabulary also influences the emotional response and usability by establishing clarity and predictability — users learn to recognize the cues embedded in the design vocabulary, which guides their interactions and builds trust.
- By formalizing the vocabulary, teams avoid inconsistent or conflicting design decisions that could confuse users or degrade brand value. It also speeds up the design and development processes by providing a ready reference framework that enhances communication within multidisciplinary teams.
- However, design vocabulary evolves over time to adapt to new trends, technologies, and user needs, reflecting a dynamic balance between consistency and innovation.

2. What are the allowable structural patterns?

Allowable structural patterns denote the predefined, widely accepted ways in which components or modules of a system (whether software, architectural, or design-related) can be organized and related to each other structurally. These patterns offer tried-and-tested blueprints for arranging systems to enhance modularity, maintainability, scalability, and flexibility.

In software engineering, structural design patterns specifically define how classes and objects can be composed to form larger systems that are understandable and reusable. Common structural patterns include:

- Adapter Pattern: Provides a bridge between incompatible interfaces, allowing existing components to work together without changing their code.
- Bridge Pattern: Decouples an abstraction from its implementation so that the two can vary independently, which is essential for managing complexity in large systems.
- Composite Pattern: Enables building tree-like structures where individual objects and compositions of objects are treated uniformly, essential for hierarchical object representations.
- Decorator Pattern: Adds responsibilities to objects dynamically, useful for extending functionalities without modifying existing code.
- Facade Pattern: Offers a simplified interface to a suite of complex subsystems,



K. J. Somaiya College of Engineering, Mumbai-77

making them easier to use and reducing dependencies.

- Flyweight Pattern: Reduces memory footprint by sharing common parts of objects, beneficial when many objects are similar.
- Proxy Pattern: Controls access to another object, adding a layer for security, caching, or logging.

These structural patterns are “allowable” because they follow best practices that help solve common architectural challenges. They provide developers and designers with a "pattern language" to craft systems that avoid pitfalls like tight coupling, redundancy, and inflexibility.

Beyond software, structural patterns can be applied to physical design, organizational systems, and user interaction flows, serving as a universally helpful concept wherever parts must be combined systematically.

3. What is the underlying computational model?

The underlying computational model is the fundamental conceptual framework or abstraction that defines how computation is structured and carried out in a system or programming paradigm. It determines the basic building blocks of computation (such as data, memory, and instructions), how they interact, and the rules that govern their execution.

Several major computational models exist, each suited for different problem domains and technological contexts:

- Turing Model: Based on the theoretical Turing machine, this is an abstract computational engine that defines problems solvable by algorithms. It assumes infinite memory (a tape) and sequential processing. This model underpins classical algorithm theory and computability.
- von Neumann Model: Reflects the architecture of most physical computers today. It features a single memory shared for both program instructions and data, a central processing unit (CPU), and a sequential instruction execution cycle. It's simple and effective but suffers from the “von Neumann bottleneck,” limiting instruction/data throughput.
- Dataflow Model: Rather than following an explicit sequence of instructions, execution is driven by the availability of data inputs. This model naturally supports parallelism and is often used in streaming computing and reactive systems.
- Functional (Applicative) Model: Represents computation as the evaluation of mathematical functions without side effects or mutable state. This model



K. J. Somaiya College of Engineering, Mumbai-77

supports declarative programming styles like functional programming, which enables easier reasoning about code and parallel execution.

- **Object-Oriented Model:** Organizes computation as interactions among encapsulated objects that contain both data and behaviors. This abstraction maps well to real-world entities and supports modularity and inheritance.
- **Logic Programming Model:** Uses formal logic to express computation, where programs consist of facts and rules, and computation is performed via inference. This is the foundation of languages like Prolog.

Selecting the right computational model shapes the entire architecture, programming practices, and types of applications that can be efficiently implemented. Each model offers strengths (e.g., simplicity, parallelism, modularity) and limitations (e.g., complexity, performance bottlenecks) affecting how computation is conceptualized and executed.

4. What are the advantages and limitations of using a particular style?

Design Vocabulary

- **Advantages:**
 - Establishes a consistent, unified identity across all product touchpoints, enhancing brand recognition and trust.
 - Reduces cognitive load for users by providing familiar visual cues and interaction patterns, improving usability and satisfaction.
 - Speeds up product development by providing a reusable set of design elements, workflows, and guidelines.
 - Facilitates collaboration among designers, developers, and stakeholders by creating a common language and reference points.
- **Limitations:**
 - Can limit creativity if the vocabulary is overly rigid or prescriptive, stifling innovation and differentiation.
 - May lead to monotony or over-standardization, making products feel generic or lack uniqueness.
 - Needs regular updating to stay relevant with evolving user needs and technological advances, which requires continuous investment.

Structural Patterns

- **Advantages:**
 - Encourage best practices in system architecture, improving code quality, modularity, and ease of maintenance.
 - Help manage complexity by breaking down systems into manageable,



K. J. Somaiya College of Engineering, Mumbai-77

well-understood parts.

- Increase reusability and scalability, allowing systems to adapt and grow with changing requirements.
- Facilitate communication and documentation by providing a shared vocabulary for common design problems.
- Limitations:
 - Overusing or misapplying patterns can introduce unnecessary complexity and decrease performance.
 - Patterns sometimes impose constraints that may not perfectly fit all problem contexts, requiring customization.
 - Learning and mastering many patterns can be challenging for new developers, possibly leading to misuse.

Computational Model

- Advantages:
 - a. Provides a clear conceptual foundation that guides the architecture and implementation of systems.
 - b. Influences the choice of programming paradigms, languages, and hardware, optimizing performance and expressiveness.
 - c. Enables formal analysis and verification of computational processes, improving reliability.
- Limitations:
 - a. Each model has inherent trade-offs: Turing machines are theoretical and not practical for real-time systems; von Neumann architecture has speed limitations; functional models can be less intuitive for side-effect-heavy applications.
 - b. Adopting a model inappropriate to the problem domain hampers efficiency, scaling, and maintainability.
 - c. Some models complicate debugging and optimization due to abstract or non-traditional execution flows.

5. What are the style's specializations?

Specializations refer to distinct ways in which a design style is commonly modified or extended to address particular challenges or requirements more effectively. For example:



K. J. Somaiya College of Engineering, Mumbai-77

- The **Layered** architectural pattern can be adapted into specific forms such as the Model-View-Controller (MVC) framework used in user interface applications or the Service-Facade pattern for structuring business logic services.
- The **Pipes and Filters** style is often customized to fit data processing workflows like Extract, Transform, Load (ETL) pipelines where data flows sequentially through a series of transformational steps.
- The **Publish/Subscribe** pattern is specialized into approaches like event sourcing or Command Query Responsibility Segregation (CQRS), which manage complex state changes, auditing, and event tracking in distributed systems.
- **Microservices** architecture evolves into implementations that emphasize domain-driven design with bounded contexts, as well as the use of API gateways for managing communication and security across multiple services.

These specializations serve as practical guidelines for tailoring broad architectural styles to meet the specific needs of different domains or address key non-functional requirements such as security, traceability, or real-time responsiveness.